

Project Brief

- The Goals: Trigger ICMP, TCP handshake, understand HTTP/HTTPS differences, cover TLS and OCSP to better explain HTTPS
- The Tools: CMD and WireShark
- The Method: ping, tracert, curl http://example.com, curl https://google.com, netstat to check connectivity
- Methodology: CMD to give commands, ScreenShots to document and WireShark to capture the protocols “red handed”

ICMP – Part 1 - PING

- Step 1: pinging google.com in the CMD as administrator
- 1st thing that happened is a DNS request to find Google's server IP:
142.250.75.174
- Ping results: the packet contains 32 bytes of data (standard for ping), took 16-18 milliseconds per ping.
- **TTL=128** (standard)-**13** (routers on the way to Google)=**115**

```
Administrator: שורת הפקודה
C:\Windows\System32>ping google.com

Pinging google.com [142.250.75.174] with 32 bytes of data:
Reply from 142.250.75.174: bytes=32 time=16ms TTL=115
Reply from 142.250.75.174: bytes=32 time=18ms TTL=115
Reply from 142.250.75.174: bytes=32 time=16ms TTL=115
Reply from 142.250.75.174: bytes=32 time=16ms TTL=115

Ping statistics for 142.250.75.174:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 16ms, Maximum = 18ms, Average = 16ms
```

01 ping.pcapng

Help Tools Wireless Telephony Statistics Analyze Capture Go View Edit File

dns

	Info	Length	Protocol	Destination	Source	Time	
	Standard query 0x0e91 A google.com	70	DNS	192.168.1.1	192.168.1.5	0.763589900	5
...	Standard query response 0x0e91 A google.com A 142.250.75.174 NS ns4.google.com NS ns2.google.com NS ns3.google	334	DNS	192.168.1.5	192.168.1.1	0.779226700	6

ICMP – Part 1 – PING (WireShark)

icmp							
	Info	Length	Protocol	Destination	Source	Time	
	Echo (ping) request id=0x0001, seq=12/3072, ttl=64 (reply in 8)	74	ICMP	142.250.75.174	192.168.1.5	0.791995200	7
	Echo (ping) reply id=0x0001, seq=12/3072, ttl=115 (request in 7)	78	ICMP	192.168.1.5	142.250.75.174	0.808468200	8
	Echo (ping) request id=0x0001, seq=13/3328, ttl=64 (reply in 10)	74	ICMP	142.250.75.174	192.168.1.5	1.800859000	9
	Echo (ping) reply id=0x0001, seq=13/3328, ttl=115 (request in 9)	78	ICMP	192.168.1.5	142.250.75.174	1.819265400	10
	Echo (ping) request id=0x0001, seq=14/3584, ttl=64 (reply in 12)	74	ICMP	142.250.75.174	192.168.1.5	2.815875700	11
	Echo (ping) reply id=0x0001, seq=14/3584, ttl=115 (request in 11)	78	ICMP	192.168.1.5	142.250.75.174	2.832314000	12
	Echo (ping) request id=0x0001, seq=15/3840, ttl=64 (reply in 18)	74	ICMP	142.250.75.174	192.168.1.5	3.828002300	17
	Echo (ping) reply id=0x0001, seq=15/3840, ttl=115 (request in 17)	78	ICMP	192.168.1.5	142.250.75.174	3.844222800	18

- We can see in the WireShark the 4 replies from Google (142.250.75.174) with TTL=115
- Note that the Google server received 4 ping requests (Echo) and sent back 4 replies. In lines 7, 9, 11, 17 the router 192.168.1.5 sent the ping request with a large TTL of 64 (standard w=Windows/Linux default) and Google replied with a TTL of 115. Meaning the actual “hops” took 13 routers
- Also note that the “seq” (sequence number) of each request and response matches
- Finally, the “Length” column represents the size of the packet traveling. The packet grew from 32 bytes sent to 74 (request) and 78 (reply). This is due to the addition of headers describing the data in the packet and the protocols needed to make the journey (Ethernet, IP, ICMP). Therefore the packet “grew” in size when it was wrapped in an envelope for the delivery

ICMP – Part 2 – Tracert

- Step 2– tracert command – mapping the road to Google
- The 1st thing we see is that it took 8 hops through routers and 13 when pinged. This is due to **Asymmetric Routing**, meaning the return traffic might differ according to the router's decision (the next best hop) or Google using different service providers along the path (ISP)
- Hop 1 is still in the local network. Hops 2-4 are in my ISP network. Hops 5-8 occur inside Google's network until the designated IP address [142.250.75.174]
- Each hop is tested 3 times for reliability. Typically the next hop takes longer because it is “deeper” into the internet

```
C:\Windows\System32>tracert google.com
```

```
Tracing route to google.com [142.250.75.174]  
over a maximum of 30 hops:
```

1	4 ms	2 ms	42 ms	InnboxG84 [192.168.1.1]
2	7 ms	2 ms	2 ms	100.64.248.1
3	13 ms	10 ms	11 ms	10.100.0.93
4	18 ms	10 ms	10 ms	45.80.88.197
5	15 ms	12 ms	13 ms	72.14.221.92
6	19 ms	13 ms	13 ms	142.251.231.41
7	17 ms	12 ms	12 ms	142.251.240.183
8	17 ms	13 ms	14 ms	lctlvq-ad-in-f14.1e100.n et [142.250.75.174]

```
Trace complete.
```

ICMP – Part 2 – Tracert (Wireshark)

•How Tracert Works Using TTL:

Tracert is clever — it sends packets with **intentionally low TTL values**, forcing each router to send back a "Time Exceeded" error

- Each router on the path receives the command to send a ping to 142.250.75.174 and a TTL that is too low to complete the task. The routers kills the request due to TTL exceeded, but the original router remembers the routers in the path to Google

icmp											
icmp											
icmpv6											

ICMP – Part 2 – Tracert (Wireshark)

- At the bottom of the WireShark screenshot we can see where tracert switches from "Time Exceeded" back to normal ping replies. This means that tracert has finally found the route to the Google server.
- The ping reply from Google, like we saw in the ping, takes 13 hops (TTL=115)

Time-to-live exceeded (Time to live exceeded in transit) 138	ICMP	192.168.1.5	72.14.221.92	32.974678100	250
Echo (ping) request id=0x0001, seq=31/7936, ttl=6 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	43.087248900	279
Time-to-live exceeded (Time to live exceeded in transit) 114	ICMP	192.168.1.5	142.251.231.41	43.106358200	280
Echo (ping) request id=0x0001, seq=32/8192, ttl=6 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	43.107437800	281
Time-to-live exceeded (Time to live exceeded in transit) 114	ICMP	192.168.1.5	142.251.231.41	43.120936000	282
Echo (ping) request id=0x0001, seq=33/8448, ttl=6 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	43.122268100	283
Time-to-live exceeded (Time to live exceeded in transit) 114	ICMP	192.168.1.5	142.251.231.41	43.135807600	284
Echo (ping) request id=0x0001, seq=34/8704, ttl=7 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	53.240365400	302
Time-to-live exceeded (Time to live exceeded in transit) 138	ICMP	192.168.1.5	142.251.240.183	53.257723400	303
Echo (ping) request id=0x0001, seq=35/8960, ttl=7 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	53.258703800	304
Time-to-live exceeded (Time to live exceeded in transit) 138	ICMP	192.168.1.5	142.251.240.183	53.271367400	305
Echo (ping) request id=0x0001, seq=36/9216, ttl=7 (no response found!) 106	ICMP	142.250.75.174	192.168.1.5	53.272312700	306
Time-to-live exceeded (Time to live exceeded in transit) 138	ICMP	192.168.1.5	142.251.240.183	53.284949300	307
Echo (ping) request id=0x0001, seq=37/9472, ttl=8 (reply in 346) 106	ICMP	142.250.75.174	192.168.1.5	63.384210500	345
Echo (ping) reply id=0x0001, seq=37/9472, ttl=115 (request in 345) 110	ICMP	192.168.1.5	142.250.75.174	63.401764000	346
Echo (ping) request id=0x0001, seq=38/9728, ttl=8 (reply in 348) 106	ICMP	142.250.75.174	192.168.1.5	63.403031100	347
Echo (ping) reply id=0x0001, seq=38/9728, ttl=115 (request in 347) 110	ICMP	192.168.1.5	142.250.75.174	63.415956100	348
Echo (ping) request id=0x0001, seq=39/9984, ttl=8 (reply in 350) 106	ICMP	142.250.75.174	192.168.1.5	63.416928500	349
Echo (ping) reply id=0x0001, seq=39/9984, ttl=115 (request in 349) 110	ICMP	192.168.1.5	142.250.75.174	63.431294000	350

TCP – Part 1 – Curl and HTTP

- The command: `Curl http://example.com`
- Curl creates a connection to the website forcing a 3-way handshake (TCP protocol)
- The basic HTML of example.com is downloaded and presented due to curl.
- HTTP is always on port 80. my CPU listens in port 54333

```
C:\Windows\System32\cmd.exe curl http://example.com
<!doctype html><html lang= en ><head><title>Exam
dth, initial-scale=1"><style>body{background:#ee
1{font-size:1.5em}div{opacity:0.8}a:link,a:visit
p>This domain is for use in documentation exampl
href="https://iana.org/domains/example">Learn m
```

tcp.port == 80 ip.addr == 172.66.147.243									
	Info	Length	Protocol	Destination	Source	Time			
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 54333	66	TCP	172.66.147.243	192.168.1.5	1.751503200	9		
	Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM WS=8192 [SYN, ACK] 54333 → 80	70	TCP	192.168.1.5	172.66.147.243	1.763911100	10		
	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 80 → 54333	54	TCP	172.66.147.243	192.168.1.5	1.764046100	11		
	GET / HTTP/1.1	129	HTTP	172.66.147.243	192.168.1.5	1.764208200	12		
	Seq=1 Ack=76 Win=131072 Len=0 [ACK] 54333 → 80	58	TCP	192.168.1.5	172.66.147.243	1.775889100	13		
	Seq=1 Ack=76 Win=131072 Len=837 [TCP PDU reassembled in 15] [PSH, ACK] 54333 → 80	895	TCP	192.168.1.5	172.66.147.243	1.781333500	14		
	HTTP/1.1 200 OK (text/html)	63	HTTP	192.168.1.5	172.66.147.243	1.781333500	15		
	Seq=76 Ack=843 Win=64512 Len=0 [ACK] 80 → 54333	54	TCP	172.66.147.243	192.168.1.5	1.781437100	16		
	Seq=76 Ack=843 Win=64512 Len=0 [FIN, ACK] 80 → 54333	54	TCP	172.66.147.243	192.168.1.5	1.782099800	17		
	Seq=843 Ack=77 Win=131072 Len=0 [FIN, ACK] 54333 → 80	58	TCP	192.168.1.5	172.66.147.243	1.794467300	18		
	Seq=77 Ack=844 Win=64512 Len=0 [ACK] 80 → 54333	54	TCP	172.66.147.243	192.168.1.5	1.794583700	19		

- Filter applied in WS: `tcp.port == 80 || ip.addr == 172.66.147.243 (http://example.com)`
- Lines 9-11: SYN, SYN-ACK, ACK sequence is captured between me and <http://example.com>)
- Line 10: The simple GET command of HTTP is captured (triggered by **curl**)

TCP – Part 2 – Curl and HTTPS

```
C:\Windows\System32>curl https://google.com_
```

- The command: Curl https://google.com
- The results in WireShark:

tcp									
	Info	Length	Protocol	Destination	Source	Time			.No
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 64124	66	TCP	142.250.75.174	192.168.1.5	0.044531500	3		
	Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64124 → 443	70	TCP	192.168.1.5	142.250.75.174	0.059348100	4		
	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 443 → 64124	54	TCP	142.250.75.174	192.168.1.5	0.059516400	5		
	Client Hello (SNI=google.com) 511		TLSv1.3	142.250.75.174	192.168.1.5	0.068278300	6		
	Seq=1 Ack=458 Win=269312 Len=0 [ACK] 64124 → 443	58	TCP	192.168.1.5	142.250.75.174	0.081819300	7		
	Server Hello, Change Cipher Spec 1470		TLSv1.3	192.168.1.5	142.250.75.174	0.135962200	8		
	Seq=1413 Ack=458 Win=269312 Len=1412 [TCP PDU reassembled in 11] [PSH, ACK] 64124 → 443	1470	TCP	192.168.1.5	142.250.75.174	0.135962200	9		
	Seq=2825 Ack=458 Win=269312 Len=1412 [TCP PDU reassembled in 11] [ACK] 64124 → 443	1470	TCP	192.168.1.5	142.250.75.174	0.135962200	10		
	Application Data 1001		TLSv1.3	192.168.1.5	142.250.75.174	0.135962200	11		
	Seq=458 Ack=5180 Win=65280 Len=0 [ACK] 443 → 64124	54	TCP	142.250.75.174	192.168.1.5	0.136131100	12		
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 64125	66	TCP	142.250.75.35	192.168.1.5	0.217783700	15		
	Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64125 → 80	70	TCP	192.168.1.5	142.250.75.35	0.232659600	16		
	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 80 → 64125	54	TCP	142.250.75.35	192.168.1.5	0.232844400	17		
gsr1/MFEwTzBNMEswSTAJBgUrDgMCGgUABBS3V7W2nAf4FiMTjpDJKg%2BMgGgMQQUYHtmGkUN18qJUC99BM00qP%2F8%2FUsCEHe9DWz	293		HTTP	142.250.75.35	192.168.1.5	0.233119600	18		
	Seq=1 Ack=240 Win=269568 Len=0 [ACK] 64125 → 80	58	TCP	192.168.1.5	142.250.75.35	0.244983200	19		
	Seq=1 Ack=240 Win=269568 Len=1412 [TCP PDU reassembled in 21] [ACK] 64125 → 80	1470	TCP	192.168.1.5	142.250.75.35	0.246955000	20		
	Response 735		OCSF	192.168.1.5	142.250.75.35	0.246955000	21		
	Seq=240 Ack=2090 Win=65280 Len=0 [ACK] 80 → 64125	54	TCP	142.250.75.35	192.168.1.5	0.247066600	22		
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 64126	66	TCP	142.250.75.67	192.168.1.5	0.289284600	25		
	Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64126 → 80	70	TCP	192.168.1.5	142.250.75.67	0.302906100	26		
	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 80 → 64126	54	TCP	142.250.75.67	192.168.1.5	0.303079200	27		
wr2/MFEwTzBNMEswSTAJBgUrDgMCGgUABBRQtSEi8EX%2BbYUTXd8%2ByMxD3s1zQQU3hse7XkV1D43JMMhu%2Bw0W1CsJACEENKnOkx	291		HTTP	142.250.75.67	192.168.1.5	0.303288700	28		
	Seq=1 Ack=238 Win=269568 Len=0 [ACK] 64126 → 80	58	TCP	192.168.1.5	142.250.75.67	0.315259600	29		
	Response 1172		OCSF	192.168.1.5	142.250.75.67	0.317134300	30		
	Change Cipher Spec, Application Data 134		TLSv1.3	142.250.75.174	192.168.1.5	0.342192800	31		
	Application Data 150		TLSv1.3	142.250.75.174	192.168.1.5	0.342612900	32		
	Seq=5180 Ack=538 Win=269312 Len=0 [ACK] 64124 → 443	58	TCP	192.168.1.5	142.250.75.174	0.357865500	33		
	Seq=5180 Ack=634 Win=269312 Len=0 [ACK] 64124 → 443	58	TCP	192.168.1.5	142.250.75.174	0.357865500	34		
	Seq=238 Ack=1115 Win=64256 Len=0 [ACK] 80 → 64126	54	TCP	142.250.75.67	192.168.1.5	0.359426000	35		

HTTPS – Drill Down – TCP/TLS Handshake TLS

- Filter: ip.addr==142.250.75.174 (Google)
- Lines 3-5 = The regular TCP handshake (on **PORT 443**)
- Line 6 = Unlike HTTP, HTTPS adds a security layer. The client (my PC) suggests TLSv1.3 for encryption
- Line 8 = Google accepts the offer and sets cipher to TLSv1.3
- Lines 9-11 = keys are being exchanged
- Line 12 = ACK from Google – saying “I’m ready to begin”

ip.addr==142.250.75.174

	Info	Length	Protocol	Destination	Source	Time
	Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 443 → 64124	66	TCP	142.250.75.174	192.168.1.5	0.044531500 3
	Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64124 → 443	70	TCP	192.168.1.5	142.250.75.174	0.059348100 4
	Seq=1 Ack=1 Win=65280 Len=0 [ACK] 443 → 64124	54	TCP	142.250.75.174	192.168.1.5	0.059516400 5
	Client Hello (SNI=google.com)	511	TLSv1.3	142.250.75.174	192.168.1.5	0.068278300 6
	Seq=1 Ack=458 Win=269312 Len=0 [ACK] 64124 → 443	58	TCP	192.168.1.5	142.250.75.174	0.081819300 7
	Server Hello, Change Cipher Spec	1470	TLSv1.3	192.168.1.5	142.250.75.174	0.135962200 8
	Seq=1413 Ack=458 Win=269312 Len=1412 [TCP PDU reassembled in 11] [PSH, ACK] 64124 → 443	1470	TCP	192.168.1.5	142.250.75.174	0.135962200 9
	Seq=2825 Ack=458 Win=269312 Len=1412 [TCP PDU reassembled in 11] [ACK] 64124 → 443	1470	TCP	192.168.1.5	142.250.75.174	0.135962200 10
	Application Data	1001	TLSv1.3	192.168.1.5	142.250.75.174	0.135962200 11
	Seq=458 Ack=5180 Win=65280 Len=0 [ACK] 443 → 64124	54	TCP	142.250.75.174	192.168.1.5	0.136131100 12

HTTPS – Drill Down - OCSP

- Lines 15-35 = Different Google IP's participates in the task (curl command). Each commences their own 3-way TCP handshake and TLS handshake
- Lines 20-21 and 29-30 = **OCSP = Online Certificate Status Protocol**

The PC is checking: "Is Google's SSL certificate still valid? Has it been revoked?"

This is an **automatic security check that happens with every HTTPS connection**

- Lines 18 and 28 – HTTP redirect.

These are **certificate chain verifications** — Google proving its identity through trusted certificate authorities.

Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 64125 66	TCP	142.250.75.35	192.168.1.5	0.217783700	15
Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64125 → 80 70	TCP	192.168.1.5	142.250.75.35	0.232659600	16
Seq=1 Ack=1 Win=65280 Len=0 [ACK] 80 → 64125 54	TCP	142.250.75.35	192.168.1.5	0.232844400	17
JBgUrDgMCGGuABBS3V7W2nAf4fIMTjpDJKg6%2BMgGqMQQUYHtmGkUNl8qJUC99BM00qP%2F8%2FUsCEHe9DWz 293	HTTP	142.250.75.35	192.168.1.5	0.233119600	18
Seq=1 Ack=240 Win=269568 Len=0 [ACK] 64125 → 80 58	TCP	192.168.1.5	142.250.75.35	0.244983200	19
Seq=1 Ack=240 Win=269568 Len=1412 [TCP PDU reassembled in 21] [ACK] 64125 → 80 1470	TCP	192.168.1.5	142.250.75.35	0.246955000	20
Response 735	OCSP	192.168.1.5	142.250.75.35	0.246955000	21
Seq=240 Ack=2090 Win=65280 Len=0 [ACK] 80 → 64125 54	TCP	142.250.75.35	192.168.1.5	0.247066600	22
Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM [SYN] 80 → 64126 66	TCP	142.250.75.67	192.168.1.5	0.289284600	25
Seq=0 Ack=1 Win=65535 Len=0 MSS=1412 SACK_PERM WS=256 [SYN, ACK] 64126 → 80 70	TCP	192.168.1.5	142.250.75.67	0.302906100	26
Seq=1 Ack=1 Win=65280 Len=0 [ACK] 80 → 64126 54	TCP	142.250.75.67	192.168.1.5	0.303079200	27
3gUrDgMCGGuABBRQtSEi8EX%2BbYUTXd8%2ByMxD3s1zQQU3hse7Xkv1D43JMMhu%2Bw00W1CsJACEENKnOKx 291	HTTP	142.250.75.67	192.168.1.5	0.303288700	28
Seq=1 Ack=238 Win=269568 Len=0 [ACK] 64126 → 80 58	TCP	192.168.1.5	142.250.75.67	0.315259600	29
Response 1172	OCSP	192.168.1.5	142.250.75.67	0.317134300	30
Change Cipher Spec, Application Data 134	TLSv1.3	142.250.75.174	192.168.1.5	0.342192800	31
Application Data 150	TLSv1.3	142.250.75.174	192.168.1.5	0.342612900	32
Seq=5180 Ack=538 Win=269312 Len=0 [ACK] 64124 → 443 58	TCP	192.168.1.5	142.250.75.174	0.357865500	33
Seq=5180 Ack=634 Win=269312 Len=0 [ACK] 64124 → 443 58	TCP	192.168.1.5	142.250.75.174	0.357865500	34
Seq=238 Ack=1115 Win=64256 Len=0 [ACK] 80 → 64126 54	TCP	142.250.75.67	192.168.1.5	0.359426000	35

HTTP vs. HTTPS Comparison

<u>STEP</u>	<u>HTTP (example.com)</u>	<u>HTTPS (google.com)</u>
DNS lookup	YES	YES
TCP handshake	3 packets	3 packets
TLS handshake	None	YES
OCSP check	None	Certified validation
Data readable?	Fully readable	Encrypted
Port used	80	443

HTTP: TCP handshake → GET → response (simple, fast, naked)

HTTPS: TCP handshake → TLS handshake → OCSP check
→ certificate verification → encrypted GET
→ encrypted response (complex, secure, private)

Netstat -an

The command: **netstat -an**:

A **snapshot** of every active network connection and open port on the PC at this exact moment

What we see:

1) **LISTENING**: Mostly open doors to Windows

PORTS (0.0.0.0: 135)

2) **ESTABLISHED**: Mostly loopback connections, the PC talking to itself (127.0.0.1: 51916) and https (443 port)

3) **TIME_WAIT**

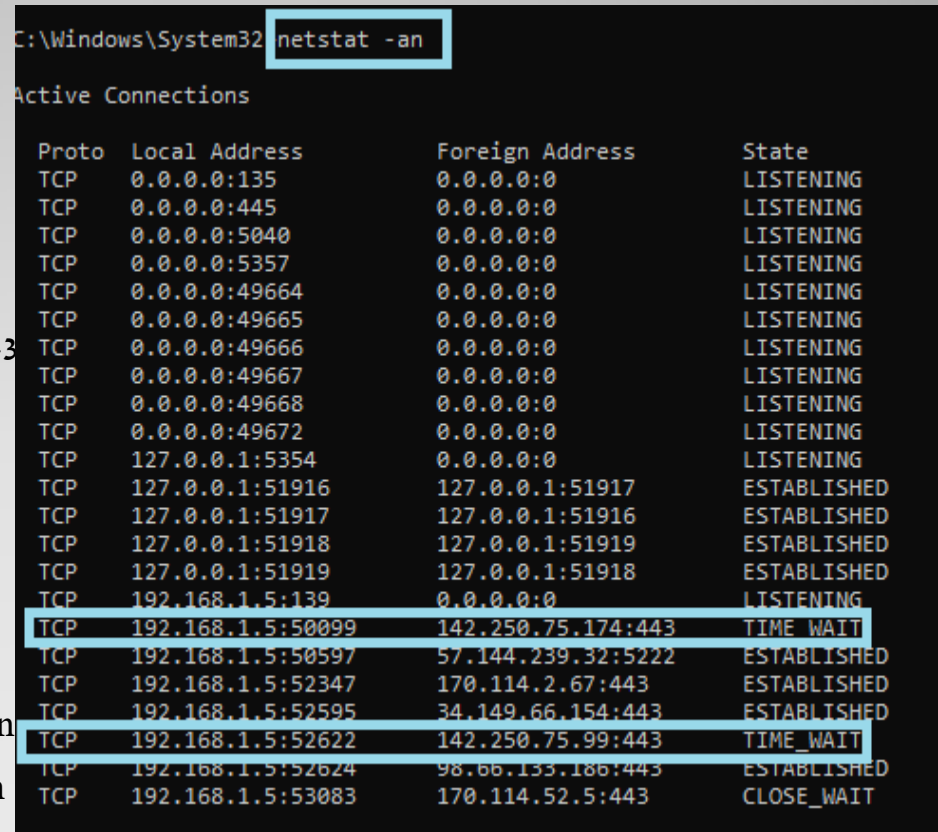
142.250.75.174: 443

142.250.75.174: 443

Those are the connection we opened to the Google Server dying down. The TCP is holding the port open briefly to catch any delayed packets still in transition

4) **CLOSE_WAIT** — Connection Being Torn Down.

The server already sent FIN and closed it's side



```
C:\Windows\System32 netstat -an
```

Active Connections

Proto	Local Address	Foreign Address	State
TCP	0.0.0.0:135	0.0.0.0:0	LISTENING
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING
TCP	0.0.0.0:5357	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49664	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49665	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49666	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49667	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49668	0.0.0.0:0	LISTENING
TCP	0.0.0.0:49672	0.0.0.0:0	LISTENING
TCP	127.0.0.1:5354	0.0.0.0:0	LISTENING
TCP	127.0.0.1:51916	127.0.0.1:51917	ESTABLISHED
TCP	127.0.0.1:51917	127.0.0.1:51916	ESTABLISHED
TCP	127.0.0.1:51918	127.0.0.1:51919	ESTABLISHED
TCP	127.0.0.1:51919	127.0.0.1:51918	ESTABLISHED
TCP	192.168.1.5:139	0.0.0.0:0	LISTENING
TCP	192.168.1.5:50099	142.250.75.174:443	TIME_WAIT
TCP	192.168.1.5:50597	57.144.239.32:5222	ESTABLISHED
TCP	192.168.1.5:52347	170.114.2.67:443	ESTABLISHED
TCP	192.168.1.5:52595	34.149.66.154:443	ESTABLISHED
TCP	192.168.1.5:52622	142.250.75.99:443	TIME_WAIT
TCP	192.168.1.5:52624	98.66.133.166:443	ESTABLISHED
TCP	192.168.1.5:53083	170.114.52.5:443	CLOSE_WAIT

THANK YOU FOR WATCHING